Deep Learning y Agentes Inteligentes

Guía de laboratorio

DriveGuard Lab

Detección de somnolencia y riesgo del conductor



Curso:	Deep Learning y Agentes Inteligentes
Docente:	Ing. Julián René Muñoz Burbano
Proyecto:	DriveGuard Lab v1.0 (continuación de SentimentLab V3)
Duración:	4 sesiones de 2 horas
Modalidad:	Individual o en parejas
Entorno:	Anaconda · Jupyter · Streamlit · FastAPI · MongoDB

Aviso de seguridad

Este sistema es un **prototipo académico**. No sustituye sistemas de seguridad vehicular certificados. Todas las pruebas se realizan frente al computador, con el vehículo detenido o con videos grabados. **Nunca** lo pruebe mientras conduce en vía pública.

Índice

1. Introducción **2**

2. Objetivos **2**

3. Marco teórico **2**

3.1. Puntos faciales con MediaPipe FaceLandmarker 2
3.2. Eye Aspect Ratio (EAR) 3
3.3. Mouth Aspect Ratio (MAR) y bostezos 3
3.4. PERCLOS 3
3.5. Orientación de la cabeza 4
3.6. Motor de riesgo 4
3.7. Expresiones faciales: ViT y CNN 4
3.8. Arquitectura multihilo y alarma local 4
3.9. Protocolo de emergencia 5

4. Arquitectura del sistema **5**

5. Materiales y entorno **6**

6. Instalación **6**

7. Procedimiento **7**

7.1. Práctica 1 — Verificación del entorno (sesión 1, 40 min) 7
7.2. Práctica 2 — EAR, MAR, PERCLOS y motor de riesgo (sesiones 1 y 2) 7
7.3. Práctica 3 — Aplicación en vivo y protocolo de emergencia (sesión 3) 8
7.4. Práctica 4 — CNN de ojos abiertos y cerrados (sesión 4) 9

8. Preguntas de análisis **9**

9. Entregables y evaluación **10**

10. Consideraciones éticas y de seguridad **10**

11. Solución de problemas **11**

A. Estructura del proyecto **11**

B. Endpoints de la API **12**

1. Introducción

La somnolencia al volante es una de las causas de siniestros viales más difíciles de detectar: el conductor suele no darse cuenta de que está a punto de dormirse. Un *microsueño* de dos segundos a 80 km/h equivale a recorrer más de 44 m sin control del vehículo.

En este laboratorio se construye **DriveGuard Lab**, un sistema que observa al conductor con una cámara web y estima su nivel de riesgo a partir de señales visuales: cuánto tiempo mantiene los ojos cerrados, con qué frecuencia bosteza, si cabecea o gira la cabeza y qué expresión facial aparenta. Cuando detecta riesgo, hace sonar una alarma; si el conductor no responde, avisa automáticamente a sus contactos de emergencia por correo electrónico y Telegram.

El proyecto continúa *SentimentLab V3*: conserva su arquitectura (`core/`, API con FastAPI, interfaz con Streamlit, MongoDB, notebooks y pruebas) y reutiliza su modelo de expresiones faciales.

2. Objetivos

Objetivo general

Diseñar, implementar y evaluar un sistema de visión por computador que detecte señales de somnolencia y distracción en un conductor, genere alertas sonoras locales y ejecute un protocolo de emergencia, integrando modelos preentrenados, un motor de reglas, una API REST y una base de datos NoSQL.

Objetivos específicos. Al finalizar el laboratorio, el estudiante será capaz de:

1. Calcular e interpretar las métricas EAR, MAR y PERCLOS a partir de puntos faciales de MediaPipe.
2. Explicar por qué un detector de rostros (Haar Cascade) no basta para medir somnolencia.
3. Implementar un motor de riesgo con calibración por conductor, ventanas temporales e histéresis.
4. Justificar el uso de hilos separados para video, clasificación de expresiones y alarma.
5. Configurar y probar un protocolo de emergencia con notificaciones reales o simuladas.
6. Entrenar una CNN para clasificar ojos abiertos y cerrados, y compararla con la regla del EAR.
7. Evaluar el sistema con métricas (precisión, recall, F1, falsas alarmas, latencia) y analizar sus limitaciones técnicas y éticas.

3. Marco teórico

3.1. Puntos faciales con MediaPipe FaceLandmarker

FaceLandmarker es un modelo preentrenado de Google que, a partir de una imagen RGB, estima **478 puntos tridimensionales** del rostro, 52 coeficientes de expresión (*blendshapes*) y una matriz de transformación 4×4 que describe la orientación de la cabeza. Funciona en tiempo real en CPU (entre 10 y 25 ms por fotograma en un portátil).

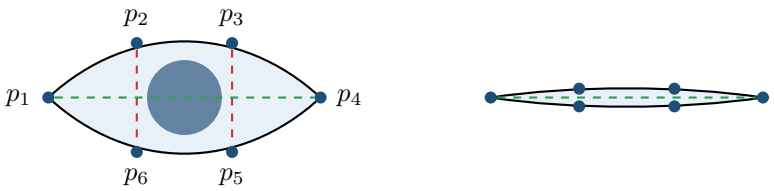
¿Por qué no basta el Haar Cascade de SentimentLab V3?

El detector Haar Cascade devuelve solo el *rectángulo* donde está la cara. Para saber si los ojos están cerrados o la boca abierta se necesita la *forma* de los párpados y los labios, es decir, puntos faciales. Por eso DriveGuard Lab usa FaceLandmarker (API *Tasks* de MediaPipe).

3.2. Eye Aspect Ratio (EAR)

Soukupová y Čech [1] proponen medir la apertura del ojo con seis puntos: dos comisuras (p_1, p_4) y dos pares de puntos en los párpados superior (p_2, p_3) e inferior (p_6, p_5):

$$\mathrm{EAR} = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2\,\|p_1 - p_4\|} \tag{1}$$



Ojo abierto: EAR ≈ 0,30 Ojo cerrado: EAR ≈ 0,05

Figura 1: Puntos del EAR. Las distancias verticales (rojo) se dividen por la horizontal (verde).

El EAR es **invariante a la escala**: no cambia si el conductor se acerca o se aleja de la cámara. Un ojo abierto da valores entre 0,25 y 0,35; uno cerrado, menos de 0,15. En la malla de MediaPipe los índices son `[33, 160, 158, 133, 153, 144]` (ojo derecho) y `[362, 385, 387, 263, 373, 380]` (ojo izquierdo).

3.3. Mouth Aspect Ratio (MAR) y bostezos

De forma análoga, la apertura de la boca se mide con las comisuras internas ($c_1 = 78$, $c_2 = 308$) y tres pares verticales $(81, 178)$, $(13, 14)$ y $(311, 402)$:

$$\mathrm{MAR} = \frac{\frac{1}{3}\sum_{k=1}^{3}\|v_k^{\mathrm{sup}} - v_k^{\mathrm{inf}}\|}{\|c_1 - c_2\|} \tag{2}$$

Con la boca cerrada MAR ≈ 0–0,1; al hablar, 0,2–0,4. Se cuenta un **bostezo** cuando MAR $> 0{,}6$ durante al menos 2 s.

3.4. PERCLOS

PERCLOS (*PERcentage of eyelid CLOSure*) es la proporción de tiempo que los ojos permanecen cerrados en una ventana deslizante [2]. Es uno de los indicadores de somnolencia más usados:

$$\mathrm{PERCLOS} = \frac{\text{fotogramas con ojos cerrados en los últimos 60 s}}{\text{fotogramas con rostro en los últimos 60 s}} \tag{3}$$

Una persona alerta parpadea unas 15–20 veces por minuto (PERCLOS < 10 %). Valores superiores al 30 % indican cansancio importante.

3.5. Orientación de la cabeza

De la matriz de transformación R se obtienen tres ángulos: *pitch* (inclinación arriba–abajo), *yaw* (giro izquierda–derecha) y *roll*. Durante la calibración se toma la postura natural del conductor como referencia; un **cabeceo** es una inclinación de más de 15° respecto a esa referencia y una **distracción**, un giro de más de 30°.

3.6. Motor de riesgo

El motor combina los indicadores en cuatro niveles (tabla 1).

Tabla 1: Reglas por defecto del motor de riesgo (configurables en la aplicación).

Indicador	Regla	Nivel
Microsueño	ojos cerrados de forma continua $\geq 1{,}5\,$s	ALTO
Cabeza caída	inclinación $> 15°$ durante $\geq 3\,$s	ALTO
Cansancio (PERCLOS)	> 30 % del último minuto con ojos cerrados	MEDIO
Bostezos	≥ 3 bostezos en 5 minutos	MEDIO
Cabeceos	≥ 3 cabeceos en 60 s	MEDIO
Distracción	cabeza girada $> 30°$ o rostro ausente durante $> 3\,$s	MEDIO
Expresión negativa	> 70 % de clasificaciones negativas en 30 s	BAJO

Tres mecanismos hacen que el motor sea robusto:

- **Calibración por conductor.** Durante los primeros 10 s se mide la mediana del EAR del conductor con los ojos abiertos (EAR base). El umbral de ojo cerrado es el 75 % de ese valor. Así se adapta a personas con ojos más grandes o más pequeños.
- **Suavizado.** El EAR se promedia en los últimos 3 fotogramas para reducir el ruido.
- **Histéresis.** El nivel sube de inmediato, pero solo baja tras 2 s sin la condición. Esto evita que la alarma se encienda y apague con cada fotograma.

3.7. Expresiones faciales: ViT y CNN

Se reutiliza el clasificador `trpakov/vit-face-expression` de SentimentLab V3, un *Vision Transformer* (ViT) [4] que divide la imagen en parches y los procesa con mecanismos de atención. Una **CNN**, en cambio, aplica filtros convolucionales que extraen patrones locales. En el laboratorio 04 se entrena una CNN pequeña para clasificar recortes de ojos.

Expresiones no son emociones

Las etiquetas del ViT (enojo, tristeza, miedo…) describen *apariencias* faciales, no emociones internas. El modelo tiene sesgos de su conjunto de entrenamiento. Por eso, en DriveGuard Lab, la expresión negativa solo genera un riesgo *bajo*.

3.8. Arquitectura multihilo y alarma local

`streamlit-webrtc` procesa cada fotograma en un **hilo de video** independiente del hilo de Streamlit. Desde ese hilo **no se puede** usar `st.audio` ni modificar la página. Como la aplicación se ejecuta en el mismo portátil que tiene la cámara, la alarma suena localmente con `pygame` (o

`winsound` en Windows) desde un hilo propio. El ViT, que tarda entre 150 y 600 ms en CPU, también corre en un hilo aparte y solo cada `EXPRESION_INTERVALO_S` segundos, para no frenar la detección de somnolencia.

3.9. Protocolo de emergencia

Una cámara que mira al conductor **no puede detectar un choque**. En un vehículo real eso lo hace un acelerómetro (sistemas como eCall). DriveGuard Lab define tres disparadores (figura 2):

1. **Conductor que no responde:** nivel ALTO y 10 s más con ojos cerrados, cabeza caída o sin rostro visible.
2. **Botón SOS:** envío inmediato.
3. **Choque simulado:** botón o archivo CSV de acelerómetro con un pico mayor que 4 g.

En los casos 1 y 3 se muestra una cuenta regresiva de 15 s con el botón *Estoy bien* para cancelar falsas alarmas.

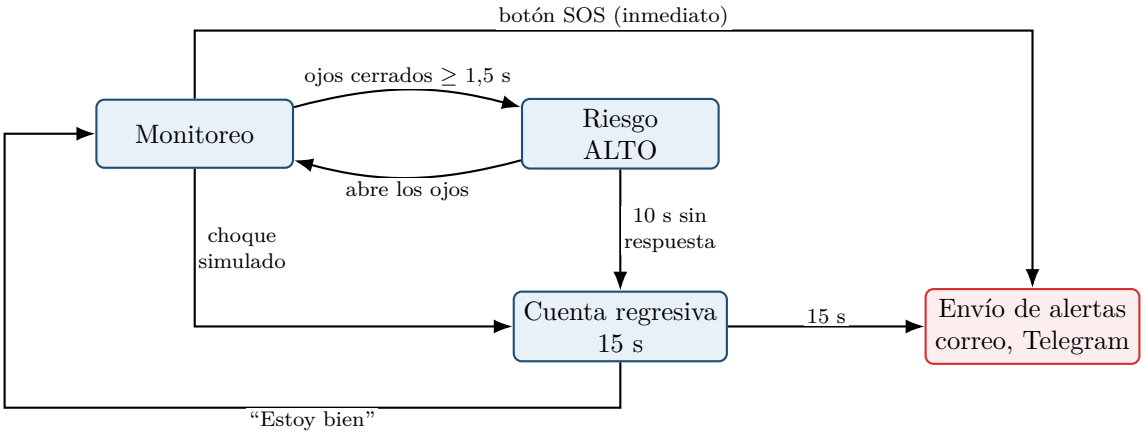


Figura 2: Estados del protocolo de emergencia.

4. Arquitectura del sistema

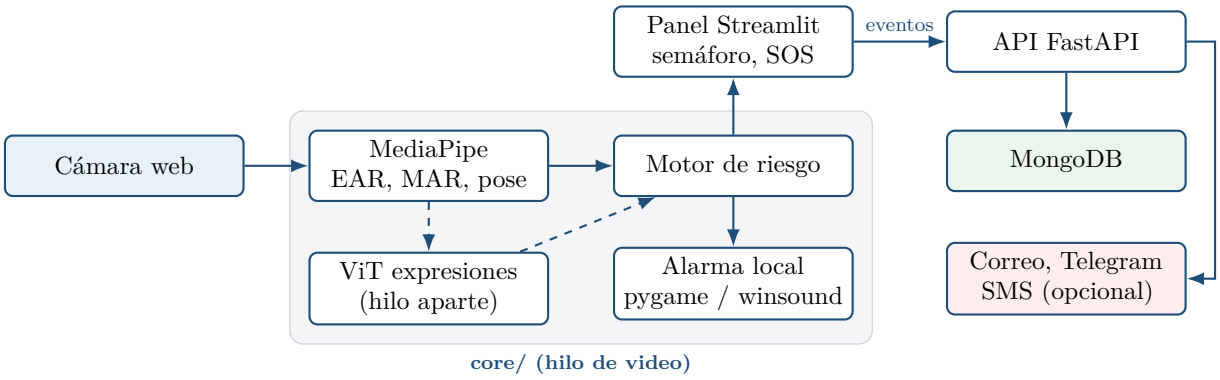


Figura 3: Componentes principales de DriveGuard Lab.

Tabla 2: Cambios respecto a SentimentLab V3 y su justificación.

Cambio	**Justificación**
MediaPipe FaceLandmarker	El Haar Cascade solo encuentra la cara; para ojos y boca se necesitan puntos faciales.
Alarma local con pygame/winsound	El hilo de video de streamlit-webrtc no puede reproducir sonido en la página.
Protocolo de emergencia	Una cámara no detecta choques: se usan conductor que no responde, SOS y choque simulado.
Creación automática de `data/`	En el laboratorio 02 la carpeta no venía en el zip y la celda de cámara fallaba.
ViT en hilo aparte cada N s	El ViT es lento en CPU; así no afecta la detección de somnolencia.

5. Materiales y entorno

- Portátil con Windows 10/11 (o macOS/Linux), 8 GB de RAM como mínimo y cámara web.
- Anaconda o Miniconda con Python 3.11.
- MongoDB Community Server (servicio local) y, opcionalmente, MongoDB Compass.
- Navegador Chrome o Edge actualizado.
- Una cuenta de Gmail con verificación en dos pasos (para el correo) y Telegram (para los mensajes).
- Proyecto `DriveGuardLab.zip` entregado por el docente.

6. Instalación

Abra **Anaconda Prompt**, descomprima el proyecto y ubíquese en su carpeta.

```
conda create -n DriveGuardLab python=3.11 -y
conda activate DriveGuardLab
python -m pip install -r requirements.txt
python -m ipykernel install --user --name DriveGuardLab --display-name "Python (
    DriveGuard Lab)"
copy .env.example .env
python scripts/descargar_modelos.py
python scripts/generar_recursos.py
python -m pytest
```

Punto de verificación

`pytest` debe terminar con **43 passed**. Si falla alguna importación, revise que el entorno `DriveGuardLab` esté activado.

Para ejecutar la aplicación se necesitan **dos consolas** con el entorno activado:

```
:: Consola 1 - API
python -m uvicorn backend.main:app --reload --port 8000

:: Consola 2 - Interfaz
python -m streamlit run frontend/app.py
```

7. Procedimiento

7.1. Práctica 1 — Verificación del entorno (sesión 1, 40 min)

1. Abra `jupyter lab` desde la raíz del proyecto y ejecute `notebooks/00_Verificacion_Entorno.ipynb` con el kernel *Python (DriveGuard Lab)*.
2. Compruebe que todas las librerías aparecen con `[OK]` y que el modelo `face_landmarker.task` se descargó.
3. Cambie `SONAR = True` y escuche los dos sonidos de alarma.
4. Cambie `USE_CAMERA = True` y tome una captura. Observe que se guarda en `data/capturas/` sin errores: la carpeta se crea automáticamente.
5. Verifique que MongoDB responde. Si no, inicie el servicio en `services.msc`.

Punto de verificación

Captura guardada, sonidos audibles y mensaje `[OK] MongoDB responde`.

7.2. Práctica 2 — EAR, MAR, PERCLOS y motor de riesgo (sesiones 1 y 2)

Trabaje en `notebooks/03_Somnolencia_EAR_MAR.ipynb`.

1. **Secciones 2–4.** Ejecute las celdas de EAR y MAR. Verifique a mano, con la ecuación (1), el EAR del ojo abierto de la figura del notebook.
2. **Sección 5.** Ejecute el motor con los siete escenarios sintéticos. Registre en la tabla 3 el nivel máximo y el instante en que se activa cada alerta.
3. **Sección 6.** Ejecute el experimento de umbrales y complete la tabla 4. Elija la configuración que considere más adecuada y justifíquela.
4. **Sección 7.** Tome una foto suya y visualice los 478 puntos. Anote su EAR con los ojos abiertos, entrecerrados y cerrados.
5. **Sección 8.** Grabe un video de 60–90 s con este guion:
   a) 0–10 s: mirar al frente (calibración).
   b) 10–25 s: parpadeo normal.
   c) 25–30 s: ojos cerrados 3 s.
   d) 35–45 s: un bostezo largo.
   e) 50–60 s: tres cabeceos.
   f) 65–75 s: girar la cabeza a un lado 5 s.

   Analícelo y compare las alertas detectadas con lo que realmente hizo.

Tabla 3: Registro de escenarios sintéticos (complete).

Escenario	Nivel máximo	Instante (s)	Motivo	¿Emergencia?
normal				
cansancio				
microsueño				
sin_respuesta				
distracción				
cabeceos				
rostro_ausente				

Tabla 4: Experimento de umbrales: detección / falsas alarmas (%).

microsueno_s	factor 0,65	factor 0,75	factor 0,85
0,5 s	/	/	/
1,0 s	/	/	/
1,5 s	/	/	/
2,0 s	/	/	/

7.3. Práctica 3 — Aplicación en vivo y protocolo de emergencia (sesión 3)

1. Inicie la API y la interfaz. En la barra lateral deben aparecer *API conectada* y *MongoDB conectado*.
2. **Conductor.** Registre su nombre y al menos un contacto de emergencia (un compañero) con correo y `chat_id` de Telegram.
3. **Notificaciones.** Configure Gmail y Telegram en `.env` (ver README, sección 7). Reinicie la API y use *Configuración* → *Enviar mensaje de prueba*. Si no tiene credenciales, active `NOTIFICACIONES_SIMULADAS=true`.
4. **Monitoreo en vivo.** Pulse *START*, mire al frente durante la calibración y observe el semáforo. Pruebe, en este orden:
   a) parpadeos normales (debe permanecer en NORMAL);
   b) ojos cerrados 2 s (ALTO y sirena);
   c) *Estoy bien* (se silencia la alarma);
   d) tres bostezos largos (MEDIO);
   e) girar la cabeza 4 s (distracción);
   f) ojos cerrados más de 12 s (conductor que no responde y cuenta regresiva). Deje que termine la cuenta regresiva y confirme que el compañero recibe el mensaje.
5. **SOS y choque.** Pulse *SOS*. Luego abra *Simulación de impacto*, cargue el archivo `acelerometro_choque.csv` de la carpeta `data/ejemplos` y cancele la cuenta regresiva con *Estoy bien*. Repita con `acelerometro_sin_choque.csv`.
6. **Historial.** Revise la sesión: eventos por tipo, gráficas de EAR y PERCLOS y alertas enviadas. Exporte los eventos a CSV.

7. **Rendimiento.** Anote los FPS con las expresiones activadas y desactivadas, y con `EXPRESION_INTERVALO_S` = 1, 2 y 5 s.

Punto de verificación

El contacto recibe el mensaje con nombre, hora, motivo, nivel y ubicación. El historial muestra la sesión con sus eventos y las alertas enviadas. Una segunda alerta en menos de 2 minutos aparece como *omitida_cooldown*.

Tabla 5: Registro de pruebas en vivo (complete).

Prueba	¿Detectó?	Nivel	Latencia (s)	Observaciones
Parpadeo normal				
Ojos cerrados 2 s				
Bostezos				
Giro de cabeza				
Sin respuesta 12 s				
SOS				
Choque simulado				

7.4. Práctica 4 — CNN de ojos abiertos y cerrados (sesión 4)

Trabaje en `notebooks/04_CNN_Ojos_Abiertos_Cerrados.ipynb`.

1. Descargue el MRL Eye Dataset [3] y descomprímalo en `data/ojos/mrl/`, o capture sus propios ojos con la sección 2 del notebook (`USE_CAMERA = True`).
2. Entrene la CNN (sección 5) y registre la curva de pérdida y la exactitud de validación.
3. Evalúe en el conjunto de prueba: exactitud, F1 por clase y matriz de confusión.
4. Mida el tiempo de inferencia por fotograma.
5. Grabe un video con intervalos conocidos de ojos cerrados y compare EAR vs. CNN (sección 8).
6. Active la CNN en la aplicación con `OJOS_METODO=cnn` y repita la prueba de microsueño.

Modo demostración

Si no hay datos, el notebook genera ojos sintéticos solo para comprobar que el código funciona. Esas métricas no son válidas para el informe ni para usar la CNN en la aplicación.

8. Preguntas de análisis

1. ¿Por qué el EAR es invariante a la distancia entre el conductor y la cámara? Demuéstrelo a partir de la ecuación (1).
2. ¿Qué ocurre con el EAR de una persona con gafas oscuras? ¿Y de noche sin iluminación infrarroja?
3. Explique el compromiso entre detección y falsas alarmas que observó en el experimento de umbrales. ¿Qué error es más grave en este contexto y por qué?

4. ¿Por qué la histéresis evita que la alarma “parpadee”? Dé un ejemplo numérico.
5. ¿Por qué no se puede reproducir el sonido de la alarma con `st.audio` desde el procesador de video? ¿Qué cambiaría si la cámara estuviera en otro equipo?
6. ¿Qué ventajas y desventajas tiene calibrar el umbral por conductor frente a un umbral fijo?
7. ¿Qué riesgos tiene enviar alertas automáticas? Proponga dos medidas para reducir las falsas alertas.
8. Compare EAR y CNN en exactitud, velocidad, necesidad de datos y explicabilidad.
9. ¿Por qué una expresión facial “negativa” no indica necesariamente que el conductor esté en riesgo?
10. ¿Qué sensores adicionales usaría en un vehículo real para detectar un siniestro?

9. Entregables y evaluación

Entregables:

1. Notebooks 03 y 04 ejecutados, con las tablas de esta guía completas.
2. Capturas de pantalla de la aplicación: semáforo en cada nivel, cuenta regresiva, historial y mensaje recibido por el contacto.
3. Archivo CSV de eventos de una sesión en vivo.
4. Informe (máximo 6 páginas) con resultados, respuestas a las preguntas de análisis y conclusiones.

Tabla 6: Rúbrica de evaluación.

Criterio	Descripción	Peso
Funcionamiento	Entorno instalado, aplicación en vivo, notificaciones y CNN funcionando.	30 %
Evaluación con métricas	Tablas completas: detección, falsas alarmas, latencia, FPS, F1 y matriz de confusión.	30 %
Análisis crítico	Explicación de errores, compromiso de umbrales, limitaciones y respuestas a las preguntas.	25 %
Documentación	Informe claro, capturas, código comentado y referencias.	15 %
	Total	**100 %**

10. Consideraciones éticas y de seguridad

- **Consentimiento:** toda persona que aparezca en la cámara debe autorizarlo.
- **Privacidad:** el sistema no guarda imágenes, solo métricas. Proteja el acceso a MongoDB, que contiene datos de contacto.
- **Alertas:** use contactos de prueba que sepan que recibirán mensajes. Nunca envíe alertas a servicios de emergencia reales.
- **Expresiones:** no use las etiquetas de expresión para evaluar, calificar ni tomar decisiones sobre personas.
- **Seguridad vial:** no pruebe el sistema mientras conduce. Un prototipo puede fallar.

11. Solución de problemas

Síntoma	Solución
API no disponible	Inicie uvicorn desde la raíz del proyecto (consola 1).
MongoDB no disponible	Inicie el servicio *MongoDB Server* o revise `MONGO_URI`.
No se pudo iniciar el detector	Ejecute `python scripts/descargar_modelos.py`.
La cámara no inicia	Permita la cámara en el navegador y cierre Teams, Zoom u otras apps.
No suena la alarma	Pruebe `ALARMA_BACKEND=winsound` y revise el volumen.
Muchas falsas alarmas	Recalibre con buena luz frontal; ajuste `factor_ear` o `microsueno_s`.
Error de autenticación en Gmail	Use una contraseña de aplicación, no la contraseña normal.
Telegram: *chat not found*	El contacto debe escribir primero al bot.
Video lento	Suba `EXPRESION_INTERVALO_S` o desactive las expresiones.

Referencias

[1] T. Soukupová y J. Čech, "Real-Time Eye Blink Detection using Facial Landmarks," en *21st Computer Vision Winter Workshop*, 2016.

[2] W. W. Wierwille y L. A. Ellsworth, "Evaluation of driver drowsiness by trained raters," *Accident Analysis & Prevention*, vol. 26, no. 5, pp. 571–581, 1994.

[3] R. Fusek, "Pupil localization using geodesic distance," en *Advances in Visual Computing (ISVC 2018)*, Springer, 2018. (MRL Eye Dataset.)

[4] A. Dosovitskiy *et al.*, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," en *ICLR*, 2021.

[5] Google, "MediaPipe Face Landmarker," https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker.

A. Estructura del proyecto

```
DriveGuardLab/
|-- core/        config, landmarks, riesgo, pipeline, alarma, faces,
|                ojos_cnn, notificaciones, impacto, simulacion, storage
|-- backend/     main, conductores_api, sesiones_api, alertas_api, schemas
|-- frontend/    app, monitoreo_ui, conductor_ui, video_ui, historial_ui, config_ui
|-- notebooks/   00_Verificacion_Entorno, 03_Somnolencia_EAR_MAR,
|                04_CNN_Ojos_Abiertos_Cerrados
|-- scripts/     descargar_modelos, generar_recursos, datos_demo
|-- tests/       43 pruebas (pytest)
|-- docs/        esta guía
|-- assets/  data/  models/
```

B. Endpoints de la API

Método	Ruta	Descripción
GET	`/health`	Estado de la API, MongoDB y canales
POST/GET	`/conductores`	Crear y listar conductores
GET/PUT/DELETE	`/conductores/{id}`	Consultar, actualizar o eliminar
POST/GET	`/sesiones`	Crear y listar sesiones
PUT	`/sesiones/{id}/cerrar`	Cerrar sesión y calcular resumen
POST	`/eventos/lote`	Guardar eventos en lote
GET	`/estadisticas/{sesion_id}`	Resumen de una sesión
POST	`/alertas/emergencia`	Protocolo de emergencia
POST	`/alertas/prueba`	Mensaje de prueba
GET	`/alertas/telegram/chats`	Obtener chat_id de Telegram